Lab session 6

Targets

- Implement in Java a Domain model expressed as a UML class diagram
- Add functionality to the entity classes
- Verify functionality with unit tests

Tasks over Carworkshop base

Get the initial skeleton

Download from the virtual campus the file ***ri.cws_base.jpa.2026.p6.skeleton.zip***. Uncompress it into a folder. It contains a maven root project with subprojects. If you use Eclipse import it using the option *Maven/Existing maven projects*.

Implement the domain model

Work on the *cws_services_jpa* subproject. Add the domain model classes in the package `uo.ri.cws.domain`.

Following the guidelines given in the related theory session (slides available on the virtual campus), implement in Java the Domain model of the CarWorkshop application. You have the UML diagram in */docs* folder.

Please, follow this workflow:

- Implement all the entities, for each one add just:
    * natural attributes,
    * constructor(s)
    * all the getters, but just the needed setters
    * *hasCode()* and *equals()* over the attributes that define the identity
    * *toString()*
- Review the (micro) value types (already implemented)
- Implement the associations and its maintenance methods. Do it one by one, testing with the specific JUnit test
    * Add to each of the associated entities the attributes needed to keep the references (accidental attributes)
    * Add the *getters*, *_getters* and *_setters* as needed (see the slides)
    * Complete the respective class in *Associations.** with the methods *link()* and *unlink()*. Pay attention to the special cases *atomicLink()* and *atomicUnlink()*.

Argument validations

- Prices and quantities must be non-negative (`>= 0`).
- Strings must not be blank, which also means they must not be `null` or empty.
- Non-primitive types (objects) must not be `null`.
- Dates are truncated to millisecond precision.

- Prices are rounded to the nearest cent.

Monetary amounts computations

- Amount calculations are performed using *MoneyCalculator* class.
- Calculations use full decimal precision.
- Final results assigned to *Money* attributes are rounded to cents.

Add functionality

Write Java code to implement this functionality:

- Compute the amount (cost) of a:
    * Substitution: *getTotalAmount()*
    * Intervention: *getTotal Amount()* composed of
      calculateLaborAmount() + *calculateSparesAmount()*
    * WorkOrder: *computeAmount()*
    * Invoice: *computeAmount()*
- Create an invoice for several work orders: Invoice *constructor*
- Allow the transition among states of an invoice and a work order. Use for this the state diagrams available in */docs* folder.
- The methods to handle the WorkOrder aggregate: *addIntervention(), removeIntervention()*.

For all then follow and verify with the companion Junit tests. The work will be finished when all the tests show green.

Tasks for the Carworkshop extended

Once you have all the previous tests running green then you can start developing the delivery version. Download from the virtual campus the *ri.**cws_extended**.jpa.2026.p6.skeleton.zip*. In the *cws_services_jpa* project skeleton, there is a folder called *test_extended*. It contains the unit tests for the domain model of the extended version (the one you will ultimately submit).

Import the new skeleton in *a new workspace* and then copy the domain model classes from the *cws_base* project into the same package of the new project. Now you have to implement the extension to the domain model and make all the new tests to pass. The /docs folder of this new project contains the UML domain model extended.